

Leçon 2 : Virgule flottante et erreurs numériques

Analyse Numérique

Séance en classe

Objectif : Comprendre comment un ordinateur stocke les nombres réels et pourquoi cela peut provoquer des catastrophes dans les calculs scientifiques.

1. Le standard IEEE 754 (virgule flottante)

Dans un ordinateur, un nombre réel est représenté en binaire avec trois composantes :

$$\text{Nombre} = (-1)^s \times (1 + \text{mantisse}) \times 2^{\text{exposant} - \text{biais}}$$

Les trois composantes

- **Le bit de signe (s) :** 0 pour positif, 1 pour négatif.
- **L'exposant (e) :** un entier décalé par un biais.
- **La mantisse (m) :** la partie fractionnaire (toujours de la forme $1, m$).

Les deux formats les plus courants

Format	Signe	Exposant	Mantisse
Simple précision (32 bits)	1 bit	8 bits	23 bits
Double précision (64 bits)	1 bit	11 bits	52 bits

Exemple : représentation de 0.3 en binaire

Nous avons vu que $0.3_{10} = 0.01001100110011 \dots_2$ (périodique). En notation scientifique normalisée :

$$0.3_{10} = 1.001100110011 \dots_2 \times 2^{-2}$$

En double précision, l'ordinateur ne peut stocker que 52 bits de la mantisse. Il tronque ou arrondit le reste. Ce petit écart est la source de l'erreur.

Pourquoi $0.2 + 0.3 \neq 0.5$ parfois ?

En réalité, 0.2 et 0.3 sont arrondis dès leur entrée dans l'ordinateur. Leur somme est donc la somme de deux approximations. Parfois, les erreurs s'annulent ; parfois, elles s'additionnent.

2. Effet des erreurs sur les méthodes numériques

Exemple 1 : La soustraction de deux nombres très proches (phénomène de cancellation)

Soit à calculer $f(x) = \sqrt{x+1} - \sqrt{x}$ pour x grand.

Pour $x = 10^6$:

$$\sqrt{10^6 + 1} - \sqrt{10^6} = 1000.0005 - 1000 = 0.0005$$

En virgule flottante, $\sqrt{10^6 + 1}$ et $\sqrt{10^6}$ sont presque égaux. La soustraction fait perdre des chiffres significatifs. Il est préférable de réécrire :

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

Morale : Les soustractions de nombres presque égaux sont dangereuses.

Exemple 2 : La méthode de Horner vs l'évaluation directe

Soit le polynôme $P(x) = (x-1)^{10}$ développé :

$$P(x) = x^{10} - 10x^9 + 45x^8 - 120x^7 + 210x^6 - 252x^5 + 210x^4 - 120x^3 + 45x^2 - 10x + 1$$

Si on évalue ce polynôme en $x = 0.999$ avec la forme développée, les grands coefficients (-252 , 210 , etc.) créent des erreurs d'arrondi. En utilisant la forme factorisée $(x-1)^{10}$, on obtient un résultat beaucoup plus précis.

Morale : La forme d'évaluation d'une expression influence la stabilité numérique.

Exemple 3 : Somme de séries alternées

Soit la série $S = \sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n}$. En théorie, $S = \ln 2 \approx 0.693147$.

Si on somme les termes en commençant par les grands et en ajoutant les petits, on peut accumuler des erreurs. Si on commence par les petits, l'erreur est moindre. L'ordre des opérations compte !

Exemple 4 : Équation quadratique classique

Soit l'équation $x^2 - 2x + 0.9999 = 0$. Les racines sont 1 ± 0.01 , soit 1.01 et 0.99 .

La formule classique :

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

si b est très grand par rapport à $4ac$, peut donner une soustraction de deux nombres très proches pour l'une des racines.

On peut utiliser la forme alternative :

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}}, \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

pour éviter la cancellation.

3. Exercices pour la séance

1. Calculez en Python / Julia les trois expressions suivantes et expliquez les résultats :

```
0.1 + 0.2 == 0.3
0.1 + 0.2 - 0.3
(0.1 + 0.2) - 0.3
```

2. Évaluez le polynôme $P(x) = (x - 1)^6$ en $x = 1.001$ en utilisant la forme développée et la forme factorisée. Comparez les résultats.
3. Calculez $\sum_{n=1}^{1000} \frac{1}{n^2}$ en ajoutant les termes de 1 à 1000 et de 1000 à 1. Comparez avec $\pi^2/6 \approx 1.644934$.

4. Résumé à retenir

- Les nombres ne sont pas stockés exactement.
- Les erreurs s'accumulent.
- La soustraction de termes proches est dangereuse.
- L'ordre des opérations et la forme de l'expression influencent la précision.
- Un bon algorithme doit être **stable numériquement**.

“En analyse numérique, il ne s’agit pas de trouver la bonne réponse, mais de trouver une réponse qui ne soit pas trop fausse.”